SWEN30006 Software Modelling and Design
Workshop 8: GoF Patterns (Part 2)
Adapter, Singleton, Factory

School of Computing and Information Systems
University of Melbourne
Semester 1, 2025

Completion: You *should* complete the exercises ***as a team***. To receive a workshop mark, you **need** to demonstrate your active participation during the workshop. Your tutor will observe your participation for you to be eligible to receive a mark for this workshop. The active participation include (but not limit to) present your solutions, help your teammates, and actively engage with the team. Attending the workshop with only passive participation will *not* be given a mark. See the Workshop Overview under Subject Information on the LMS for more details.

Requisite Knowledge and Tools

To complete this workshop you will need to be familiar with the following terms and concepts:

- Class Diagram
- Sequence Diagrams
- UML State Machine Diagram
- GRASP principles

We assume that you have reviewed the **Pre-Workshop Learning** in Workshop 8 (Week 9) module in LMS when commencing this workshop activity.

Workshop Objectives

In this workshop, we will be focusing on applying GoF patterns, particularly, **Adapter, Factory, and Singleton**. The exercises begin with designing a system based on provided information, and then analysing which classes should be applied with GoF patterns to solve a design problem. Finally, we will finish by implementing the system and using GoF patterns. By the end of this workshop, you should:

- Be able to apply GoF patterns to solve design problems.
- Be able to create appropriate models to represent the design.
- Be able to implement the system based on the GoF patterns.

Introduction

GoF (Gang-Of-Four) patterns are a set of design patterns that provide solutions to common software design problems. GoF patterns are specialization of the GRASP building blocks. Each GoF pattern is constructed based on a set of principle patterns to address a specific problem. For example, Adapter pattern offers Protected Variations from changing various interfaces through the use of an Indirection object that applies interfaces and Polymorphism. GoF patterns are a useful tool for software developers. These patterns can be considered as generalised solutions or templates that solve real-world problems. However, applying patterns on a trivial problem may result in overcomplicated implementation. Hence, it is important to understand the concept of the design patterns and their benefits when applying them. Below is a brief summary of three GoF patterns that we will be using in this week's exercises.

- **Adapter:** Adapter pattern is a structural pattern in GoF patterns. It provides a solution on how to link objects that have *incompatible* interfaces. Instead of directly link these objects, Adapter pattern suggests to have an 'adapter', i.e., a class that is responsible for wrapping various interfaces of these objects and providing a single interface. Hence, other objects that want to link with these objects with various interfaces are not required to know all the interfaces, but only one interface provided by the adapter.
- **(Concrete) Factory:** Concrete Factory pattern is a part of Factory Method in GoF patterns. Factory pattern is a creational pattern as it is used to control class instantiation. Creating an object can be a complex task for some problems and may require complex logics to control this creation. To achieve Low Coupling and High Cohesion in GRASP principles, Concrete Factory pattern provides a solution by defining a new class and method that are responsible for generating an object.
- **Singleton:** Singleton pattern is also a creational pattern in GoF patterns. Singleton pattern is useful when we are sure that we need only one instance of a particular class and it will have a global point of access. To address this design problem, Singleton pattern provides a solution by hiding constructor and creating a method that is responsible for instantiate the class itself in order to ensure that the class will not be instantiated from outside the class.

The System Under Discussion: MiniExpedia

This week you will be creating a simple flight searching and booking systems, called MiniExpedia. Like the real-world Expedia and other flight searching services, MiniExpedia provides a service for searching flights from various airlines with specified departure, destination, and departure date. In this workshop, MiniExpedia will retrieve flight information from airlines' services. Below are fundamental steps that the system must have:

- Users can specify information of flights (i.e., departure airport, destination airport, departure date, departure time) and the number of passengers. Users can specify **multiple routes** for searching.
- MiniExpedia sends the search request with specified information to each airline service and present the search results (i.e., departure airport, destination airport, departure date/time of that flight, arrival date/time of that flight, and airfare) from all airlines.
- Users then can select any flight from any airline in the search results for a booking.
- Before a booking, users will specify passenger details, i.e., first name, last name, and date of birth.
- Finally, MiniExpedia sends a booking request with selected flights and passenger details to the corresponding airlines. If the booking request is successful, the airline booking service will provide a confirmation code.

Airline Packages

In this workshop, three packages with javadoc (i.e., Jetstar, Qantas, and Virgin packages) are provided for artificial airline services that you will use to retrieve flights. The class diagrams for these packages are as below.

Jetstar

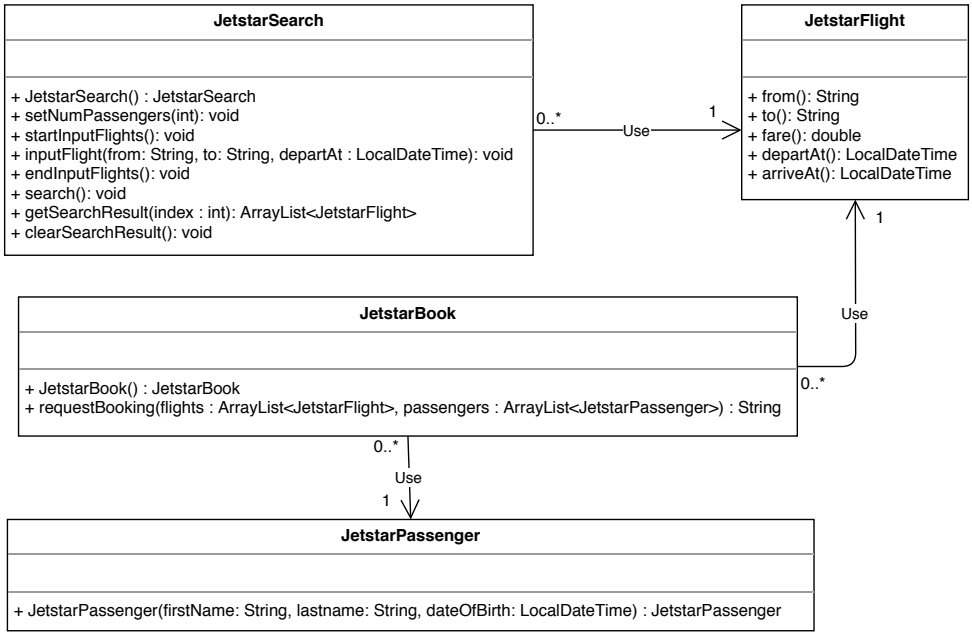

Figure 1: Class Diagram for Jetstar package.

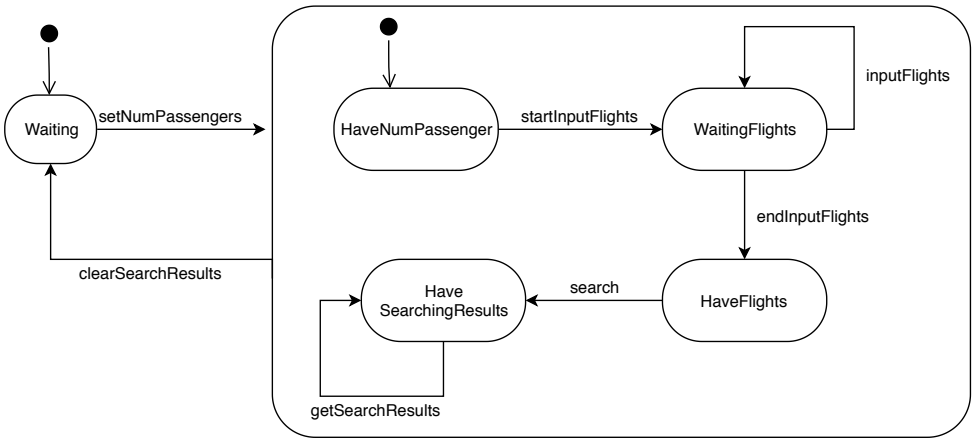

Figure 2: A State Machine Diagram for JetstarSearch class in Jetstar package. **Note:** the Jetstar classes will throw an exception message if the method is called in the invalid state.

Qantas

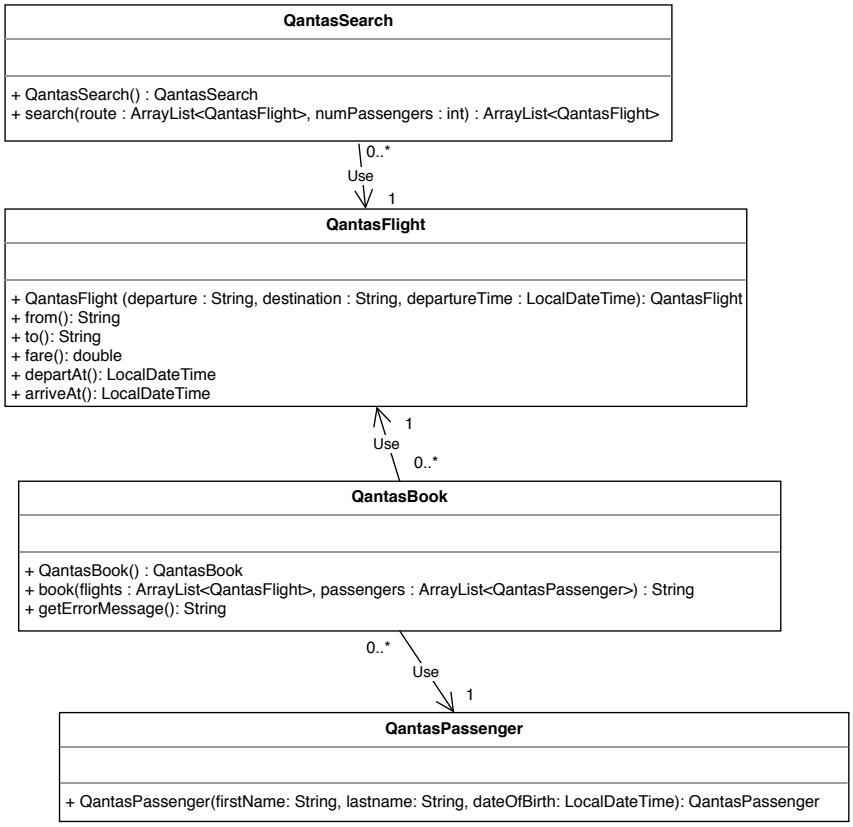

Figure 3: Class Diagram for Qantas package.

Virgin

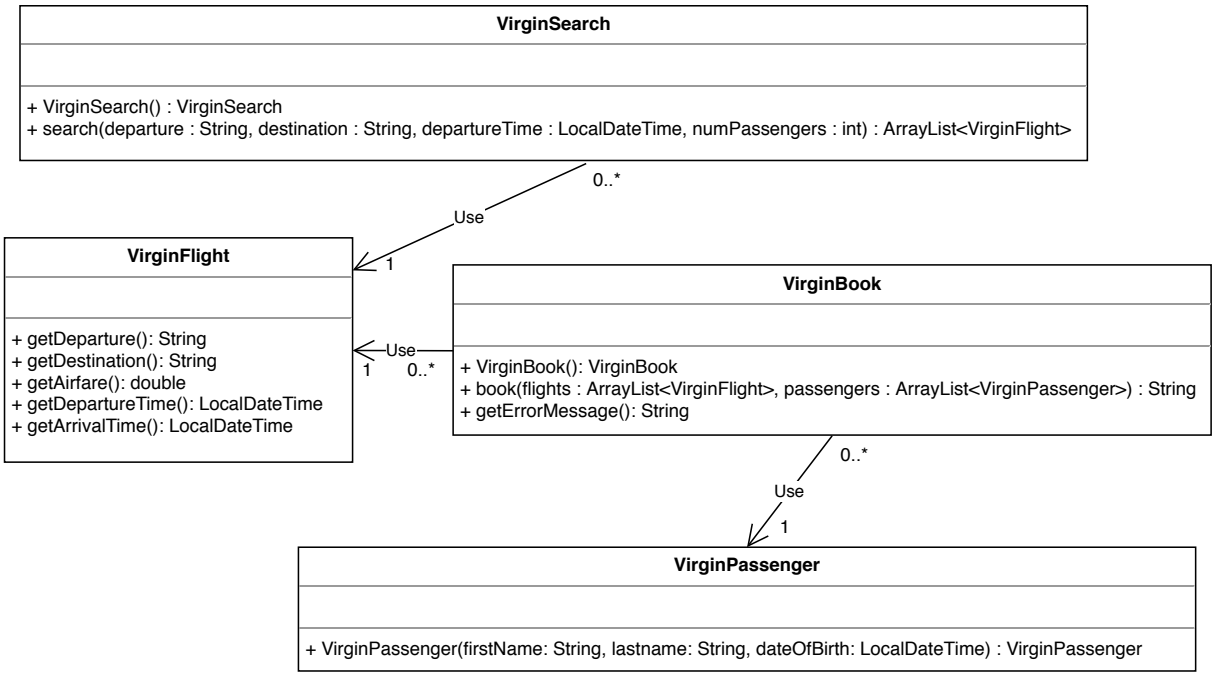

Figure 4: Class Diagram for Virgin package.

MiniExpedia Interface

An interface for MiniExpedia is provided in `MiniExpedia_interface.zip`. Below is a partial sequential diagram of how users interact with the interface.

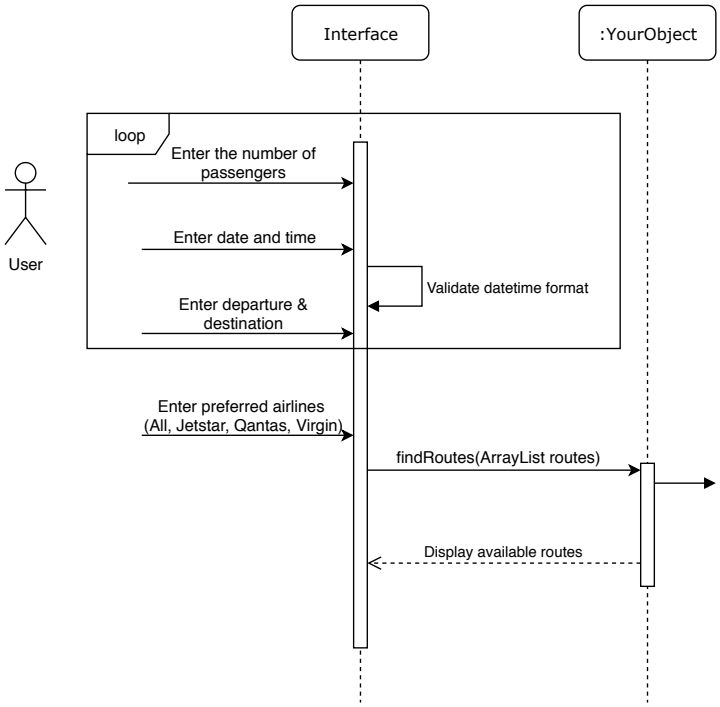

Figure 5: A partial sequential diagram of MiniExpedia.

Part 1 Designing the System

Before implementing the system, you need to design MiniExpedia in order to understand what data and classes are needed and how will MiniExpedia will interact with the external services (i.e., airline packages).

Minimum Requirement: At least two external services are used. The searching function must be designed and implemented. The design and implementation for the booking function is *optional*.

Q1.1 Creating Design Model

Using the information provided in the previous section, create a design model of your MiniExpedia.

Q1.1.1 Adapter

As you can notice in the class diagrams in the previous section, those external Search objects have different interfaces and behave differently. Identify all objects that require Adapter. Draw Class Diagrams with Adapter for all corresponding classes. Creating Class Diagrams for Adapter

Q1.1.2 Singleton Factory

To achieve High Cohesion, the Adapters in the preivous task should be created by a designated object (i.e., Factory). Draw a System Sequential Diagram to describe how your Adapters for external services will be

created and how your objects will send a request to the external services for searching and booking.

Part 2 Implementing the System

Using your design in Part 1, implement your MiniExpedia in Java. You should create the appropriate classes for Adapters and Singleton Factory. An interface to get flight information from users are provided in GitHub assignment: https://classroom.github.com/a/QXCRBdsT.

On this task, you and your team should:

- Have at least, a searching function that works
- Present how each adapter convert an interface and manipulates data for external packages.
- Present how and where do they use Singleton Factory.

You should implement code consistently with your design. By now, you should also be aware of GRASP principles and ensure that your design and implementation do not violate GRASP principles.